# SCA Security Overview

The security measures in the backend and frontend, how the frontend handles data, and the security risks that remain.

Frontend: **sca-fe** (React 19 + Vite, on Vercel) · Backend: **sca-be** (Node 20 + Express 5, on Render) · Data: **Firebase Firestore** + **Cloudflare R2** · Reviewed: 30 October 2026


1. Security at a glance
2. How data flows through the system
3. Backend security measures
4. Frontend: how it handles data
5. Frontend security measures
6. Infrastructure and secrets
7. Possible security risks
8. Recommended next steps


## 1. Security at a glance

**AES-256**
All user records and files encrypted at rest

**16**
Security risks reviewed in section 7

**4**
High risks to address

**8**
Medium risks to plan for

**Most urgent:** the Firestore security rules in `sca-be/firestore.rules` (block all direct access) were **never deployed**: the deploy failed because the Google account had no permission on the project. Check the rules in the Firebase console today (see risk R1).

## 2. How data flows through the system

| | |
|---|---|
| **Browser (React)** | Shows forms and lists. Holds the login token and the decrypted data of the signed-in user only while the page is open. Never talks to the database directly. |
| **HTTPS** | Every request goes to `https://sca-web-be.onrender.com/api/v1` over TLS (Vercel and Render both force HTTPS). |
| **API (Express)** | Checks rate limits, the login token, the role and the input; then the service **encrypts before saving** and **decrypts after reading**. |
| **Firestore** | Stores records. User-owned records are `{ reference number, data: <encrypted>, createdAt, updatedAt }`. Only the backend (Admin SDK) can reach it. |
| **Cloudflare R2** | Stores files, photos and media, encrypted on the server before upload. Storage keys are random and never sent to the browser. |

## 3. Backend security measures

| Area | What is in place |
|---|---|
| **Transport & headers** | HTTPS (Render). `helmet` sets security headers. CORS only allows the frontend origin (`CLIENT_URL`). JSON bodies limited to 100 KB. |

| | |
|---|---|
| **Authentication** | Email + password: passwords hashed with **bcrypt (12 rounds)**, 8–72 characters; email must be confirmed before sign-in. Google: the browser signs in with Firebase Auth and the backend **verifies the Firebase ID token** before issuing its own token. Sessions are **JWT** signed with `JWT_SECRET` (min 32 characters), valid for 1 day ( `JWT_EXPIRES_IN` ). |
| **Email links** | Confirmation, password reset and developer invitations use one-time random tokens. Only a **SHA-256 hash** is stored; links expire (reset 1 hour, confirmation 24 hours, invitation 72 hours) and are deleted after use. Forgot-password and resend never reveal whether an email exists. |
| **Authorization (RBAC)** | Roles `sca-1` , `sca-2` , `sca-3` , `dev` (an account can have several). Developer endpoints use `requireRole("dev")` , which reads the role from the database on every request, so removing a role works immediately. |
| **Ownership** | User data lives at `users/{userId}/...` and services only build paths from the signed-in user's id, so one user can never read another user's records (wrong id → `404` ). |
| **Input validation** | Every body, query and URL parameter is validated with **Zod** (types, lengths, formats, required fields); unknown fields are dropped. Ids must match the Firestore id format. |
| **Encryption at rest** | **AES-256-GCM** with a random IV per value ( `ENCRYPTION_KEY` ). Fully encrypted: students, cases, appointments, files info, categories, To Do board, phone, address; all files/photos/media in R2. GCM also detects tampering. Duplicate phones are found with a keyed hash (HMAC-SHA256) instead of plain text. Default rule (CLAUDE.md): **everything is encrypted unless the owner approves plain text**. |
| **Uploads** | Allowed types only (by extension), size limits (images 10 MB, documents 50 MB, media 50 MB, max 6 per report), kept in memory and encrypted before storage. Random storage keys so file names never appear in paths. |
| **Rich text** | System update HTML is **sanitized on the server** ( `sanitize-html` ): only basic formatting tags; links limited to http, https and mailto; scripts and event handlers removed. |
| **Rate limiting** | 300 requests/min per IP; 400 requests/10 min per user; sign-in 3 failed attempts/10 min; forgot password 2/3 min; register & resend 5/hour; uploads 6/10 min; support 5/10 min; invitations 20/hour. Over the limit → `429` . |
| **Errors & logging** | One error handler returns friendly messages; stack traces and internal errors are never sent to the browser. Audit logs record create/update/delete/download of students, cases, appointments and files (no content). |
| **Duplicate accounts** | One account per email and per mobile number, checked inside a transaction at sign-up. |

## 4. Frontend: how it handles data

| Data | Where it is kept | How it is handled |
|---|---|---|
| Login token (JWT) | `localStorage` (`sca_token`) | Sent as `Authorization: Bearer` on every request by the axios instance. Removed on logout, and automatically when the server answers `401` (expired or invalid) — the user is sent to the login page. |
| Profile and school details | Redux store (memory) | Loaded from `/auth/me` after sign-in; cleared on logout. Not written to browser storage. |
| Students, cases, appointments, files list | Component state (memory) | Fetched page by page (max 15) already decrypted by the server for the owner only; gone when the page closes. |
| Files, photos, media | Temporary blob URLs | Downloaded with the token (never a public link), shown via `URL.createObjectURL` and released after use. |
| Theme preference | `localStorage` | Light/dark setting only. No personal data. |
| Firebase config | Built into the app | Public by design (identifies the Firebase project for Google sign-in); it grants no database access. |
| Address lists | Memory cache | Public PSGC place lists (regions, cities, barangays). The chosen address is sent to our API, never to PSGC. |

## 5. Frontend security measures

- **No direct database access:** the browser only calls the API, so validation, encryption and access checks always run on the server.
- **Route guards:** signed-out users are sent to login; first-time users must complete their profile; developer pages are hidden unless the account has the `dev` role. (The backend checks roles too — the frontend only hides pages.)
- **Safe rendering:** React escapes all text by default. The only raw HTML shown is the system update content, which is sanitized on the server first.
- **Friendly errors only:** the axios layer turns failures into user-facing messages and never shows raw server errors.
- **Client-side validation** (email, password length, PH mobile format, required address) gives quick feedback; the server validates again.
- **Google sign-in hardening:** `Cross-Origin-Opener-Policy: same-origin-allow-popups` header and a redirect fallback so the popup flow works safely on Vercel.
- **Upload checks before sending:** type and size limits are checked in the browser and again on the server.
- **Secrets:** only public `VITE_*` values are in the frontend; no API keys or encryption keys are shipped to the browser.

## 6. Infrastructure and secrets

| Secret | Where it lives | Purpose |
|---|---|---|
| `ENCRYPTION_KEY` | Render env / local `.env` | Encrypts and decrypts all protected data. Losing it = losing the data. |
| `JWT_SECRET` | Render env / local `.env` | Signs login tokens. Changing it signs everyone out. |
| `FIREBASE_SERVICE_ACCOUNT_JSON` / `sca.json` | Render env / local file (gitignored) | Full admin access to Firestore and Firebase Auth. |

| | | |
|---|---|---|
| R2 access keys | Render env / local `.env` | Read/write the file bucket. |
| `BREVO_API_KEY` | Render env / local `.env` | Sends emails from the verified sender. |

`.env` and `sca.json` are in `.gitignore`. Dependency check (`npm audit`, production packages): backend **0** known issues; frontend **8** (4 high, 4 moderate) in `@grpc/grpc-js` (via the `firebase` package), `echarts` and `uuid`.

## 7. Possible security risks

Ordered by priority. HIGH fix soon MEDIUM plan a fix LOW accept or improve later.

| # | Risk | Level | Why it matters | Mitigation |
|---|---|---|---|---|
| **R1** | Firestore rules not deployed | HIGH | The deny-all rules were never deployed. If the database was created in *test mode*, anyone with the (public) Firebase config could read or write it directly, bypassing the API. Encrypted fields stay unreadable, but plain fields (emails, names, school details) and deletions would be exposed. | Open Firebase console → Firestore → Rules and set `allow read, write: if false;`, or deploy with an account that has access (`firebase deploy --only firestore:rules`). |
| **R2** | Service-account key exposed | HIGH | The `sca.json` private key was shared during development. Whoever has it has full admin access to Firestore and Auth. | Create a new key in Google Cloud, update Render and `sca.json`, then delete the old key. |
| **R3** | One key decrypts everything | HIGH | Encryption protects data at rest, not on a compromised server. Anyone with `ENCRYPTION_KEY` + database access can read all records. If the key is lost, all data is unrecoverable. | Keep an offline backup of the key; limit who can see Render env vars; plan key rotation (re-encrypt with a new key). |
| **R4** | Frontend dependency vulnerabilities | HIGH | 8 known issues (4 high) in production packages. | Update `firebase`, `echarts` and `uuid`; re-run `npm audit`; test Google sign-in and charts. |
| **R5** | Token in localStorage | MEDIUM | If any script injection (XSS) happened, it could read the token and act as the user for up to 1 day. | Keep HTML sanitized; add a Content Security Policy; consider an httpOnly cookie session later. |
| **R6** | No token revocation | MEDIUM | Logout only deletes the token in the browser. A stolen token keeps working until it expires (1 day). Deleting an account or removing a role does not end other sessions' basic access. | Shorter expiry, or a `tokenVersion` on the user checked on each request ("sign out everywhere"). |
| **R7** | No Content Security Policy on the frontend | MEDIUM | Vercel only sends the COOP header. A CSP would limit damage from any XSS. | Add a CSP header in `vercel.json` (allow self, the API, Firebase and PSGC). |
| **R8** | Plain-text fields remain | MEDIUM | `users.email`, `users.name`, school details, support report text and system update text are not encrypted (listed in ENCRYPTED_DATA.md "Fields to Review"). | Encrypt or approve each one; email needs an `emailHash` for login lookups. |
| **R9** | Upload type checked by extension only | MEDIUM | A harmful file renamed to `.pdf` would be accepted. Files are only returned to their owner as downloads, which limits impact. | Check file signatures (magic bytes) on upload. |
| **R10** | Developer role can be granted by any developer | MEDIUM | An existing account becomes `dev` immediately when invited, with access to all reports and reporter emails. A compromised developer account could add others. | Require owner approval or confirmation by the invited user; alert on role changes. |

| | | | | |
|---|---|---|---|---|
| **R11** | No two-factor authentication; simple password rule | **MEDIUM** | Only length (8+) is enforced. Phished or reused passwords give full access to confidential student data. | Add 2FA (e.g. email or app codes) for counselors and developers; check against common passwords. |
| **R12** | Rich text allows `style` attributes | **MEDIUM** | Scripts are removed, but inline styles could make an update look misleading (e.g. fake buttons). Only developers can publish. | Restrict allowed styles to text-align and colors. |
| **R13** | Account enumeration at sign-up | **LOW** | Registering with a used email or mobile shows "already exists", revealing that an account exists. Rate limits slow this down. | Accept, or send "check your email" for every sign-up. |
| **R14** | Rate limits are in memory | **LOW** | Counters reset when Render restarts or sleeps, and would not be shared between several servers. | Move counters to Redis if scaling out. |
| **R15** | Shared or unattended computers | **LOW** | An open session on a school computer shows decrypted student data until logout or token expiry. | Add an inactivity auto-logout; remind users to sign out. |
| **R16** | Audit logs are not reviewed | **LOW** | Actions are recorded but there is no screen or alert to notice misuse. | Add an audit view or alerts for bulk downloads and deletions. |

## 8. Recommended next steps

1. **Today:** confirm/deploy the deny-all Firestore rules (R1) and rotate the service-account key (R2).
2. **This week:** back up `ENCRYPTION_KEY` offline (R3), update vulnerable frontend packages (R4), and run `npm run db:encrypt-all` after deploying the backend so older records are encrypted.
3. **Next:** add a Content Security Policy (R7), token revocation (R6), file signature checks (R9) and owner approval for developer roles (R10).
4. **Later:** 2FA (R11), decide on the remaining plain-text fields (R8), inactivity logout (R15) and an audit log view (R16).